

AuraSearch — Technical Documentation

A high-performance, disk-persistent local search engine built on classical, deterministic computer science.

Table of Contents

1. Project Overview
 2. Design Philosophy
 3. System Architecture
 4. Core Data Structure — The Inverted Index
 5. Write-Heavy Ingestion Pipeline
 - 5.1 Directory Crawler
 - 5.2 Document Parsers
 - 5.3 The Analyzer Chain
 6. Read-Optimized Query Engine
 - 6.1 Query Analysis
 - 6.2 Relevance Scoring — TF-IDF / BM25
 - 6.3 Result Lookup
 7. Memory Management — MMap Strategy
 8. Production Resilience
 - 8.1 State Tracking
 - 8.2 Tombstone Pattern
 - 8.3 Periodic Compaction
 9. Data Flow Diagrams
 10. Performance Characteristics
 11. Constraints & Assumptions
 12. Glossary
-

1. Project Overview

AuraSearch is a local search engine designed to index and search documents on disk with sub-millisecond keyword lookup latency. It supports the following file formats:

Format	Extension	Parser
Plain Text	.txt	Direct byte read
Markdown	.md	Custom syntax stripper
Web Pages	.html	Custom tag stripper

Unlike semantic search systems that rely on neural networks and vector embeddings, AuraSearch operates entirely on classical, deterministic algorithms — trading probabilistic "meaning-matching" for exact, reproducible, lightning-fast keyword retrieval.

The system is optimized around a key principle: **do expensive work at write time, not read time**. By running an intensive text analysis pipeline during document ingestion, the query engine is left with near-trivial lookups at runtime.

2. Design Philosophy

AuraSearch is built on three core design decisions:

2.1 Write-Heavy, Read-Light

All expensive computation (parsing, normalization, stemming, index construction) happens at **ingestion time**. By the time a user fires a query, the engine does nothing more than a hash map lookup and a ranking sort.

2.2 Memory-Constrained from the Start

The system is explicitly designed to operate within an **8 GB RAM ceiling**. Rather than loading entire index structures into memory, AuraSearch uses Memory-Mapped Files (MMap) to create a virtual address space bridge between RAM and disk — maintaining near-RAM read speeds without the memory overhead.

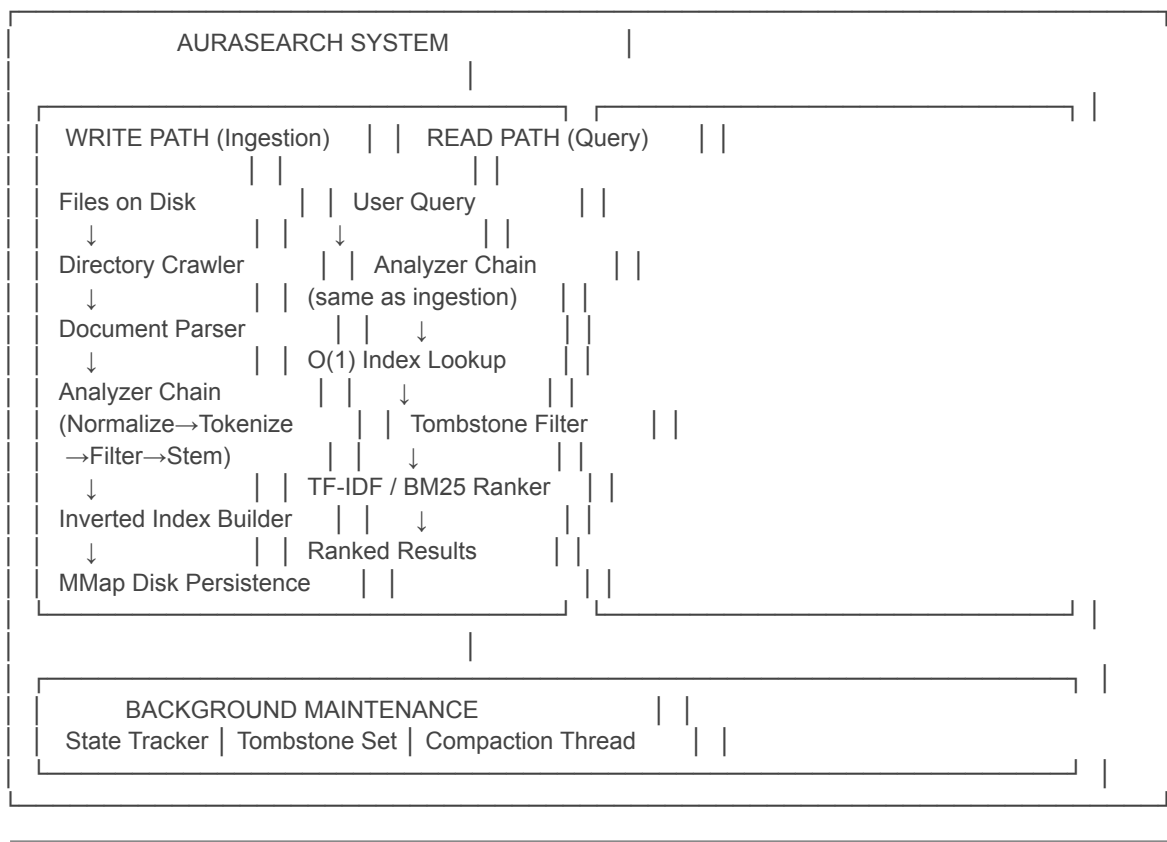
2.3 Classical CS over Neural Approaches

AuraSearch deliberately avoids embedding models, transformers, or any form of learned representation. Every algorithm in the system — from the Porter Stemmer to BM25 scoring — is mathematically deterministic. This ensures:

- Reproducible results across runs
 - No GPU requirements
 - No model loading latency
 - No dependency on third-party AI infrastructure
-

3. System Architecture

AuraSearch is split into two primary subsystems:



4. Core Data Structure — The Inverted Index

4.1 Structure

The foundational data structure of AuraSearch is an **Inverted Index** — a term borrowed from information retrieval theory, and mathematically equivalent to a librarian's ledger: instead of asking "what words are in document X?", it asks "which documents contain word X?"

```
HashMap<String, Vec<Posting>>
```

- └─ A list of all documents that contain this term
- └─ The normalized, stemmed term (the "key")

4.2 The Posting Type

Each entry in the posting list is a **Posting** struct containing:

Field	Type	Description
<code>document_id</code>	<code>DocumentID</code>	Unique identifier for the source document

term_frequency	u32	Number of times this term appears in that document
----------------	-----	--

Term: "design"

Postings: [

```
{ document_id: 4, term_frequency: 12 },
{ document_id: 9, term_frequency: 3 },
{ document_id: 17, term_frequency: 7 },
```

]

term_frequency (TF) is the input signal for relevance ranking. A document mentioning "design" 12 times is likely more relevant to a query for "design" than one mentioning it 3 times — all else being equal.

4.3 The MMap Bridge

The in-memory **HashMap** is not stored in heap-allocated RAM indefinitely. The index is **serialized into key-value blocks and written to disk**, then accessed via Memory-Mapped Files (MMap). See Section 7 for a full treatment.

5. Write-Heavy Ingestion Pipeline

The ingestion pipeline is the "Assembler" — the system responsible for converting raw files on disk into optimized index entries. It is composed of four sequential stages.

5.1 Directory Crawler

Responsibility: Discover all indexable files on the local machine.

Behavior:

- Performs a **recursive depth-first traversal** of the target directory tree.
- Filters files by extension: **.txt**, **.md**, **.html** only; all other file types are skipped.
- Handles **permission boundary errors gracefully** — if a directory is inaccessible (e.g., **/root**, system directories), the crawler logs the error and continues traversal without halting the entire ingestion run.
- Returns a list of **(file_path, document_id)** tuples for downstream processing.

Error Handling Contract:

AccessDenied(path) → log warning, skip directory, continue

NotFound(path) → log warning, skip file, continue

IoError(path) → log error, skip file, continue

No permission error should propagate to a panic or halt the pipeline.

5.2 Document Parsers

Before text can be analyzed, format-specific noise must be stripped. AuraSearch implements two custom, byte-level parsers:

HTML Stripper

Removes all HTML markup, retaining only the visible readable text.

Strips:

- All tags: `<p>`, `<div>`, `<span>`, `<a>`, `<script>`, `<style>`, etc.
- HTML attributes and their values.
- HTML comments: `<!-- ... -->`
- DOCTYPE declarations.

Retains:

- Text content between tags.
- HTML entities are decoded to their character equivalents (e.g., `&` → `&`).

Markdown Stripper

Removes Markdown syntax characters, retaining readable prose.

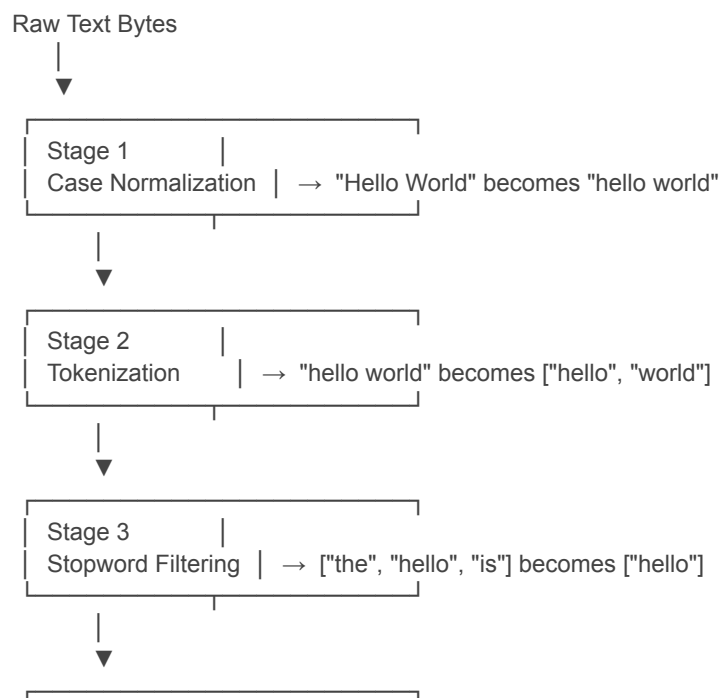
Strips:

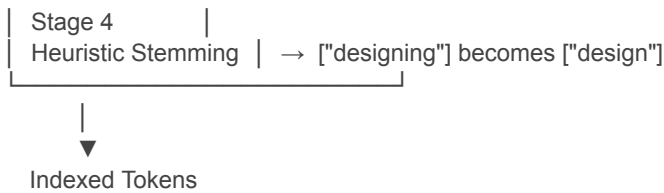
- Heading markers: `#`, `##`, `###`, etc.
- Bold/italic markers: `*`, `**`, `_`, `__`
- Link syntax: `[text](url)` — retains the link text, discards the URL.
- Image syntax: `![alt](url)` — discards entirely or retains alt text.
- Code fences: `` ` `` and inline backticks.
- Blockquote markers: `>`
- Horizontal rules: `---`, `***`

Note: Both parsers operate at the **byte level** rather than constructing a full DOM or AST. This keeps parsing fast and allocation-minimal.

5.3 The Analyzer Chain

The Analyzer Chain is the central text-processing assembly line. It takes raw text bytes as input and emits a stream of clean, normalized tokens suitable for indexing. It runs as four sequential stages:





Stage 1 — Case Normalization

All bytes in the input stream are converted to their lowercase equivalents. This ensures that "Rust", "RUST", and "rust" all map to the same index key.

Implementation note: Operates on raw bytes using ASCII-range lowercasing logic. For non-ASCII characters, a Unicode-aware lowercase function is applied.

Stage 2 — Tokenization

The normalized byte stream is split into discrete tokens (words). Splitting occurs on:

- Whitespace characters: `\t`, `\n`, `\r`
- Non-alphanumeric boundary characters: `.`, `,`, `;`, `:`, `!`, `?`, `(`, `)`, `"`, `'`, `-`, etc.

Implementation note: Uses low-level byte iteration rather than regex-based splitting to minimize allocation overhead. Tokens shorter than a minimum length threshold (e.g., 2 characters) are discarded.

Stage 3 — Stopword Filtering

Tokens that carry no informational value for search relevance are discarded via a **high-speed lookup set** — a `HashSet<str>` of approximately 150–300 common English stopwords.

Examples of filtered words: `the`, `is`, `at`, `which`, `on`, `a`, `an`, `to`, `be`, `in`, `it`, `of`, `and`, `or`, `but`, `not`, `with`, `for`, `as`, `by`, `this`, `that`.

The lookup is an $O(1)$ hash operation. The full stopwords list is statically compiled into the binary at build time, adding zero runtime loading cost.

Stage 4 — Heuristic Stemming (Porter Stemmer)

The final stage reduces tokens to their morphological root forms using the **Porter Stemmer algorithm** — a deterministic, rule-based suffix-stripping algorithm originally described by Martin Porter in 1980.

Examples:

Input Token	Stemmed Output
designing	design
indexed	index
running	run
connections	connect
relational	relat

happiness	happi
-----------	-------

Key property: The Porter Stemmer is not a lemmatizer. It does not produce valid English words — it produces *consistent root forms*. Since the same stemmer is applied to both indexed tokens and query tokens, the keys always align perfectly, regardless of whether the output is a "real" word.

6. Read-Optimized Query Engine

The Query Engine is the "Ranker" — the system responsible for taking a user's search query and returning a sorted list of relevant documents in near-zero time.

6.1 Query Analysis

When a user submits a search query, it is passed through the **exact same Analyzer Chain** used during ingestion:

User Query String

- Case Normalization
- Tokenization
- Stopword Filtering
- Porter Stemming
- Query Tokens (aligned with Index Keys)

This symmetry is the foundational guarantee of AuraSearch's correctness: because both the indexed tokens and the query tokens pass through the identical pipeline, they are always in the same normalized form. There is no mismatch risk between query intent and index state.

6.2 Relevance Scoring — TF-IDF / BM25

Documents are ranked by **relevance score**, not by insertion order or recency. AuraSearch implements a scoring algorithm based on two industry-standard models:

TF-IDF (Term Frequency — Inverse Document Frequency)

For each query term **t** and candidate document **d**:

$TF(t, d)$ = term_frequency of **t** in **d** (stored in Posting)

$IDF(t) = \log(N / df(t))$

where N = total number of documents in index

df = number of documents containing term **t**

$TF-IDF(t, d) = TF(t, d) \times IDF(t)$

Intuition:

- A word appearing frequently in a document (high TF) suggests relevance to that document.
- A word appearing across nearly all documents (low IDF, e.g., "example") is down-weighted because it's not discriminating.
- A word appearing rarely across the corpus but frequently in one document (high TF, high IDF) scores highest.

BM25 (Best Match 25)

BM25 is a probabilistic extension of TF-IDF that adds **document length normalization**, preventing longer documents from dominating results simply due to having more words.

$$\text{BM25}(t, d) = \frac{\text{IDF}(t) \times [\text{TF}(t, d) \times (k_1 + 1)]}{[\text{TF}(t, d) + k_1 \times (1 - b + b \times |d|/\text{avgdl})]}$$

where:

- k_1 = term frequency saturation parameter (typically 1.2–2.0)
- b = length normalization parameter (typically 0.75)
- $|d|$ = length of document d in tokens
- avgdl = average document length across the corpus

Multi-term queries: For queries with multiple terms, scores are summed across all query terms for each candidate document, then sorted descending.

6.3 Result Lookup

The lookup path for a single query term is:

1. Compute the stemmed query token ($O(1)$ deterministic).
2. Perform a **hash map key lookup** on the inverted index — $O(1)$ average case.
3. Retrieve the **Vec<Posting>** for that term.
4. Filter out any **document_id** values present in the active Tombstone set.
5. Score remaining postings via BM25.
6. Merge and sort across all query terms.
7. Return top- N results.

No linear scan of document content occurs at query time. The system never reads original file bytes during a search operation.

7. Memory Management — MMap Strategy

7.1 The Problem

A naive implementation that loads the entire inverted index into heap-allocated RAM would quickly exhaust the 8 GB memory ceiling when indexing thousands of dense documents. The index can grow to many gigabytes.

7.2 The Solution — Memory-Mapped Files

AuraSearch uses **Memory-Mapped Files (MMap)** as the persistence layer for the inverted index. The operating system maps a region of the file on disk directly into the process's virtual address space.

Process Virtual Address Space



Key properties:

- **Demand paging:** Only the pages of the index file actually accessed are loaded into physical RAM. The OS page cache handles this transparently.
- **No explicit serialization overhead at read time:** Reading a key-value block from the MMap region feels like a direct memory read.
- **Persistence across restarts:** The file on disk is the source of truth. The index survives process restarts with zero reload cost.
- **No GC pressure:** MMap regions bypass the application's allocator entirely. There is no garbage collection overhead for index data.

7.3 Disk Layout

The index is stored in serialized key-value block segments on disk. The layout is structured to allow the MMap layer to jump directly to any segment via known byte offsets, enabling $O(1)$ physical reads once the virtual address is resolved.

8. Production Resilience

8.1 State Tracking

A dedicated **index state tracker** monitors the state of every ingested file. It records:

- File path
- Last-modified timestamp at time of ingestion
- Associated `document_id`

On re-ingestion runs, the tracker compares current filesystem state against its records to detect:

- **New files:** Queue for ingestion.
- **Modified files:** Queue for re-ingestion; tombstone the old `document_id`.
- **Deleted files:** Tombstone the associated `document_id` immediately.

8.2 Tombstone Pattern

When a document is deleted or replaced, its index entries are **not immediately removed** from the inverted index. Instead, the `document_id` is written to a **Tombstone Set** — a `HashSet<DocumentID>` held in memory and persisted to a small sidecar file on disk.

Tombstone Set: { 4, 17, 23, 91, ... }

Why not immediate deletion?

Immediately removing all postings for a document from the inverted index would require a full scan of the index — an $O(N)$ operation proportional to the total number of posting entries. This would block the query path for an unacceptable duration on large indexes.

Tombstoning instead:

- Registering a tombstone is an **$O(1)$ set insertion**.
- The query engine checks the tombstone set during result retrieval and silently skips any `document_id` found in it — also $O(1)$ per posting.
- Disk space is reclaimed lazily during **compaction** (see below).

8.3 Periodic Compaction

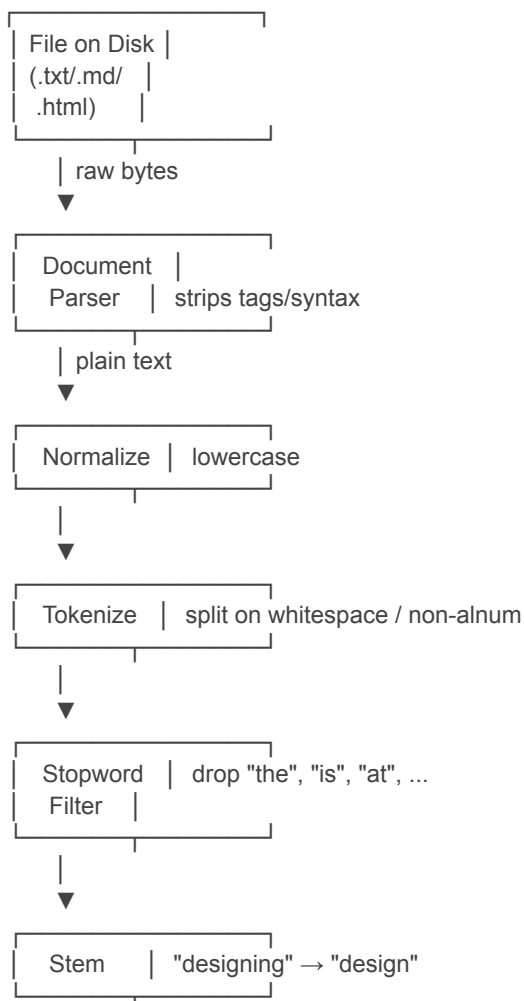
A **background compaction thread** runs periodically without blocking the main query thread. Its responsibilities:

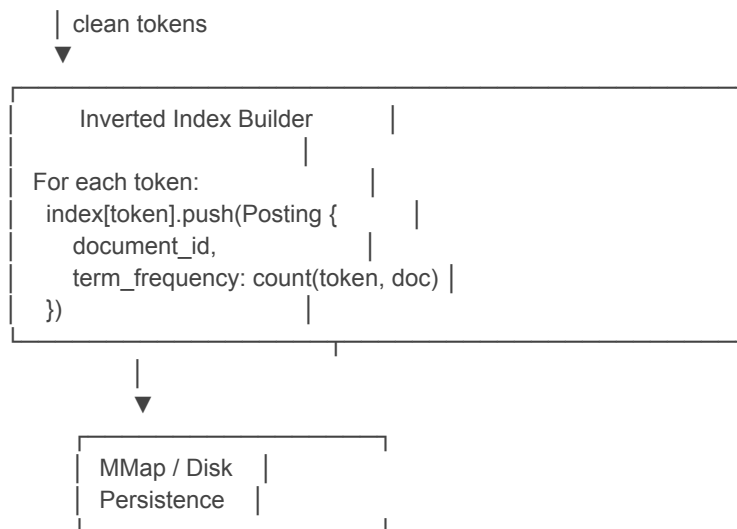
1. **Merge fragmented storage segments** accumulated from incremental writes into a single, contiguous block.
2. **Purge tombstoned document_ids** — physically remove all posting entries whose `document_id` appears in the Tombstone Set.
3. **Reclaim disk space** freed by the purge.
4. **Reset the Tombstone Set** after all purged IDs are confirmed gone from the index.

Thread safety: Compaction operates on a separate file segment copy or uses copy-on-write semantics to avoid locking the read path. Live queries continue to be served from the current (pre-compaction) MMap region until the compaction write is committed and the MMap pointer is atomically rebased.

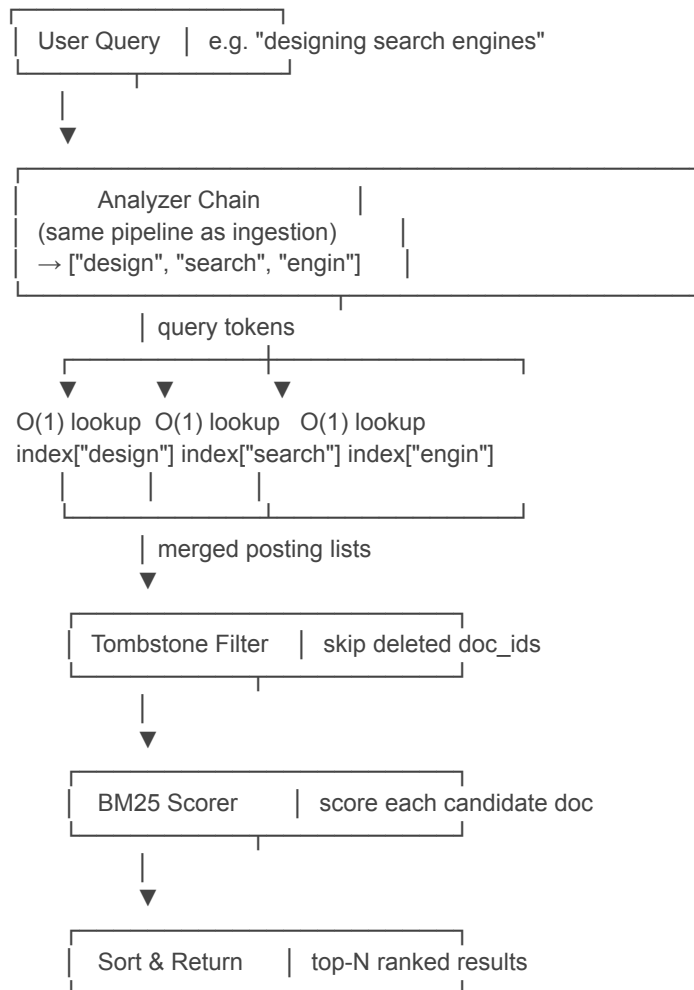
9. Data Flow Diagrams

9.1 Ingestion Flow





9.2 Query Flow



10. Performance Characteristics

Operation	Complexity	Notes
Index lookup (single term)	$O(1)$ average	HashMap key lookup
Tombstone check (per posting)	$O(1)$	HashSet membership test
BM25 scoring (per candidate doc)	$O(Q)$	Q = number of query terms
Result sort	$O(D \log D)$	D = number of candidate documents
Tombstone registration	$O(1)$	HashSet insertion
Document ingestion	$O(T)$	T = number of tokens in document
Compaction	$O(N)$	N = total posting entries; background, non-blocking
Full-text scan at query time	Never	Eliminated by design

Target latency: Sub-millisecond for single-term queries on indexes of up to tens of thousands of documents.

11. Constraints & Assumptions

Constraint	Value
RAM ceiling	8 GB
Supported file formats	<code>.txt</code> , <code>.md</code> , <code>.html</code>
Language	English (stopwords and stemmer are English-specific)
Search modality	Keyword (not semantic / vector)
Concurrency model	Single write thread; concurrent reads; background compaction thread
Index persistence	MMap-backed disk files; survives process restart

Stemmer	Porter Stemmer (heuristic, English)
---------	-------------------------------------

12. Glossary

Term	Definition
Inverted Index	A data structure mapping terms to the documents that contain them, enabling fast keyword lookup.
Posting	A record within an inverted index entry, storing a document ID and the term's frequency in that document.
Term Frequency (TF)	The number of times a given term appears in a specific document.
Inverse Document Frequency (IDF)	A measure of how rare a term is across the entire document corpus. Rare terms get higher IDF scores.
BM25	"Best Match 25" — a probabilistic ranking function that extends TF-IDF with document length normalization.
MMap (Memory-Mapped File)	An OS facility that maps a file on disk into the process's virtual address space, enabling file I/O at memory-access speeds.
Stopword	A common structural word (e.g., "the", "is") that carries no search relevance and is filtered out before indexing.
Stemming	The process of reducing a word to its morphological root form (e.g., "running" → "run").
Porter Stemmer	A classic rule-based suffix-stripping algorithm for English stemming, authored by Martin Porter (1980).
Tombstone	A marker recording that a document has been deleted, used to filter results without immediately modifying the index.
Compaction	A background maintenance process that merges index segments and physically removes tombstoned entries to reclaim disk space.

DocumentID	A unique integer identifier assigned to each ingested document.
Analyzer Chain	The sequential pipeline (normalize → tokenize → filter → stem) applied to both indexed documents and user queries.
Demand Paging	An OS behavior where pages of a memory-mapped file are loaded into physical RAM only when first accessed.